AML Transaction Monitoring – Machine Learning Anomaly Detection

Goal: detect potentially suspicious (money-laundering) transactions and **reduce false positives** compared with a traditional rules-based monitoring system.

Data: a synthetic dataset of bank transfers generated in this notebook (no real customer data). It simulates normal customer behaviour plus three common laundering typologies:

- **Structuring / smurfing** – repeated transfers just below a reporting threshold
- **Rapid fan-out** – one account quickly sending funds to many new counterparties
- **High-risk corridor layering** – round-amount transfers to high-risk jurisdictions, often at night

Approach: feature engineering on account behaviour → handling extreme class imbalance → Logistic Regression, Random Forest and Isolation Forest → evaluation with precision, recall, PR-AUC and false positives versus a rules baseline.

In [1]:
```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (precision_score, recall_score, average_precision_score
                             roc_auc_score, confusion_matrix, precision_recall_curv
rng = np.random.default_rng(42)
pd.set_option("display.float_format", "{:,.3f}".format)
```

1. Generate synthetic transaction data

In [2]:
```
N_ACC, N_NORMAL = 20_000, 600_000
HIGH_RISK = ["IR", "KP", "MM", "SY", "AF"]
LOW_RISK = ["LT", "LV", "EE", "DE", "SE", "FI", "DK", "PL", "NL", "FR", "GB", "US"]
start = pd.Timestamp("2026-01-01")

acc_profile = pd.DataFrame({"account": np.arange(N_ACC),
                            "typical_amount": rng.lognormal(5.5, 0.9, N_ACC)})

# --- normal behaviour ---
snd = rng.integers(0, N_ACC, N_NORMAL)
normal = pd.DataFrame({
    "sender": snd,
    "receiver": rng.integers(0, N_ACC, N_NORMAL),
    "amount": acc_profile.typical_amount.values[snd] * rng.lognormal(0, 0.6, N_NORM
    "country": rng.choice(LOW_RISK + HIGH_RISK, N_NORMAL, p=[0.997/12]*12 + [0.003/
```

```
    "timestamp": start + pd.to_timedelta(rng.integers(0, 90*24*3600, N_NORMAL), uni
})
# daytime-heavy hours for normal traffic
hours = rng.choice(24, N_NORMAL, p=(lambda w: w/w.sum())(np.array([1,1,1,1,1,2,4,6,
normal["timestamp"] = normal.timestamp.dt.normalize() + pd.to_timedelta(hours, unit
# some legitimate large / round / high-risk payments that fool simple rules
idx = rng.choice(N_NORMAL, 4000, replace=False)
normal.loc[idx[:2000], "amount"] = rng.uniform(9000, 9999, 2000)          # legit n
normal.loc[idx[2000:], "amount"] = rng.choice([1000, 2000, 5000, 10000], 2000).asty
normal["label"] = 0

# legitimate look-alikes that trigger rules: payroll/merchant accounts with bursts,
# importers paying high-risk-region suppliers, customers splitting large purchases
look = []
for a in rng.choice(N_ACC, 900, replace=False):
    t0 = start + pd.Timedelta(seconds=int(rng.integers(0, 85*24*3600)))
    kind = rng.integers(0, 3)
    if kind == 0:   # payroll / supplier batch: many counterparties in a few hours
        for _ in range(rng.integers(10, 30)):
            look.append([a, rng.integers(0, N_ACC), rng.lognormal(6.6, 0.5), rng.ch
                         t0 + pd.Timedelta(minutes=int(rng.integers(0, 8*60)))])
    elif kind == 1: # instalments just under 10k (e.g. car / property payments)
        for _ in range(rng.integers(3, 9)):
            look.append([a, rng.integers(0, N_ACC), rng.uniform(9000, 9990), rng.ch
                         t0 + pd.Timedelta(hours=int(rng.integers(0, 72)))])
    else:           # legitimate trade with high-risk regions, round amounts, any h
        for _ in range(rng.integers(2, 6)):
            look.append([a, rng.integers(0, N_ACC), float(rng.choice([5000, 10000,
                         rng.choice(HIGH_RISK + LOW_RISK[:3], p=[0.14]*5 + [0.1]*3)
                         t0.normalize() + pd.Timedelta(hours=int(rng.integers(0, 24
look = pd.DataFrame(look, columns=["sender", "receiver", "amount", "country", "time
normal = pd.concat([normal, look], ignore_index=True)

# --- laundering typologies ---
bad_rows = []
bad_accounts = rng.choice(N_ACC, 300, replace=False)
for a in bad_accounts:
    t0 = start + pd.Timedelta(seconds=int(rng.integers(0, 85*24*3600)))
    kind = rng.integers(0, 3)
    if kind == 0:   # structuring: 6-12 transfers just under 10k within 48h
        for _ in range(rng.integers(3, 10)):
            bad_rows.append([a, rng.integers(0, N_ACC), rng.uniform(9000, 9950), rn
                             t0 + pd.Timedelta(minutes=int(rng.integers(0, 48*60)))
    elif kind == 1: # rapid fan-out: 15-30 new counterparties within 6h
        for _ in range(rng.integers(6, 25)):
            bad_rows.append([a, rng.integers(0, N_ACC), rng.lognormal(6.3, 0.7), rn
                             t0 + pd.Timedelta(minutes=int(rng.integers(0, 10*60)))
    else:           # high-risk layering: round amounts, often at night
        for _ in range(rng.integers(2, 7)):
            ts = t0.normalize() + pd.Timedelta(hours=int(rng.choice([0,1,2,3,4,9,14
            bad_rows.append([a, rng.integers(0, N_ACC), float(rng.choice([5000, 100
                             rng.choice(HIGH_RISK + LOW_RISK[:3], p=[0.14]*5 + [0.1
bad = pd.DataFrame(bad_rows, columns=["sender", "receiver", "amount", "country", "t
bad["label"] = 1
# label noise: investigators never confirm every case
flip = rng.random(len(bad)) < 0.08
```

```
bad.loc[flip, "label"] = 0

tx = pd.concat([normal, bad], ignore_index=True).sort_values("timestamp").reset_ind
tx["amount"] = tx.amount.round(2)
print(f"{len(tx):,} transactions | suspicious: {tx.label.sum():,} ({tx.label.mean()
tx.head()
```

```
611,270 transactions | suspicious: 2,399 (0.392%)
```

Out[2]:

	sender	receiver	amount	country	timestamp	label
0	17475	14012	15,000.000	LT	2026-01-01 00:00:00	1
1	5921	4115	288.510	DK	2026-01-01 00:01:26	0
2	9138	14637	119.740	US	2026-01-01 00:01:29	0
3	6689	6885	889.360	GB	2026-01-01 00:01:35	0
4	7744	2757	59.750	PL	2026-01-01 00:01:44	0

2. Feature engineering (behavioural AML features)

In [3]:

```
tx["hour"] = tx.timestamp.dt.hour
tx["is_night"] = tx.hour.isin([0,1,2,3,4,5,22,23]).astype(int)
tx["log_amount"] = np.log1p(tx.amount)
tx["is_round"] = (tx.amount % 1000 == 0).astype(int)
tx["near_threshold"] = tx.amount.between(9000, 9999.99).astype(int)
tx["high_risk_country"] = tx.country.isin(HIGH_RISK).astype(int)

# rolling sender behaviour (past 24h / 6h windows) - the core of transaction monito
tx = tx.set_index("timestamp")
g = tx.groupby("sender")
tx["tx_count_24h"] = g["amount"].transform(lambda s: s.rolling("24h").count())
tx["amount_sum_24h"] = g["amount"].transform(lambda s: s.rolling("24h").sum())
tx["near_thr_count_48h"] = g["near_threshold"].transform(lambda s: s.rolling("48h")
tx["tx_count_6h"] = g["amount"].transform(lambda s: s.rolling("6h").count())
tx = tx.reset_index()

# deviation from the account's own normal amount
sender_median = tx.groupby("sender").amount.transform("median")
tx["amount_vs_median"] = tx.amount / sender_median
# how many different counterparties the sender used overall (network breadth)
tx["sender_unique_receivers"] = tx.groupby("sender").receiver.transform("nunique")

FEATURES = ["log_amount","hour","is_night","is_round","near_threshold","high_risk_c
            "tx_count_24h","amount_sum_24h","near_thr_count_48h","tx_count_6h",
            "amount_vs_median","sender_unique_receivers"]
tx[FEATURES + ["label"]].describe().T
```

Out[3]:

	count	mean	std	min	25%	50%	75%
log_amount	611,270.000	5.566	1.139	1.001	4.803	5.538	6.291
hour	611,270.000	12.455	4.840	0.000	9.000	12.000	16.000
is_night	611,270.000	0.092	0.289	0.000	0.000	0.000	0.000
is_round	611,270.000	0.006	0.075	0.000	0.000	0.000	0.000
near_threshold	611,270.000	0.007	0.084	0.000	0.000	0.000	0.000
high_risk_country	611,270.000	0.005	0.068	0.000	0.000	0.000	0.000
tx_count_24h	611,270.000	1.466	1.472	1.000	1.000	1.000	2.000
amount_sum_24h	611,270.000	910.499	2,704.635	1.720	149.000	325.740	726.200
near_thr_count_48h	611,270.000	0.021	0.265	0.000	0.000	0.000	0.000
tx_count_6h	611,270.000	1.218	1.253	1.000	1.000	1.000	1.000
amount_vs_median	611,270.000	1.851	10.134	0.039	0.672	1.000	1.494
sender_unique_receivers	611,270.000	31.816	6.602	10.000	27.000	31.000	35.000
label	611,270.000	0.004	0.063	0.000	0.000	0.000	0.000

3. Rules-based baseline (how traditional monitoring flags alerts)

In [4]:

```
rules = ((tx.high_risk_country == 1) | (tx.near_threshold == 1) |
         (tx.is_round & (tx.amount >= 5000)) | (tx.tx_count_24h >= 10)).astype(int)

X_train, X_test, y_train, y_test, r_train, r_test = train_test_split(
    tx[FEATURES], tx.label, rules, test_size=0.3, stratify=tx.label, random_state=4

def report(name, y_true, y_pred, score=None):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return {"model": name, "precision": precision_score(y_true, y_pred, zero_divisi
            "recall": recall_score(y_true, y_pred), "alerts": int(tp+fp), "false_po
            "PR_AUC": average_precision_score(y_true, score) if score is not None e
            "ROC_AUC": roc_auc_score(y_true, score) if score is not None else np.na

results = [report("Rules baseline", y_test, r_test)]
pd.DataFrame(results)
```

Out[4]:

	model	precision	recall	alerts	false_positives	PR_AUC	ROC_AUC
0	Rules baseline	0.130	0.697	3867	3365	NaN	NaN

4. Models – handling extreme class imbalance with class weights

In [5]:
```
lr = make_pipeline(StandardScaler(), LogisticRegression(class_weight="balanced", ma
lr.fit(X_train, y_train)
lr_score = lr.predict_proba(X_test)[:, 1]

rf = RandomForestClassifier(n_estimators=300, max_depth=12, class_weight="balanced_
                            min_samples_leaf=5, n_jobs=-1, random_state=42)
rf.fit(X_train, y_train)
rf_score = rf.predict_proba(X_test)[:, 1]

# unsupervised anomaly detection: trained without labels
iso = IsolationForest(n_estimators=300, contamination=0.01, random_state=42, n_jobs
iso.fit(X_train)
iso_score = -iso.score_samples(X_test)
```

5. Evaluation – catch the same suspicious activity with far fewer false alerts

In [6]:
```
rules_recall = recall_score(y_test, r_test)

def at_recall(score, target):
    p, r, t = precision_recall_curve(y_test, score)
    ok = np.where(r[:-1] >= target)[0]
    return t[ok[-1]]  # highest threshold that still reaches the target recall

for name, s in [("Logistic Regression", lr_score), ("Random Forest", rf_score), ("I
    thr = at_recall(s, rules_recall)
    results.append(report(f"{name} (same recall as rules)", y_test, (s >= thr).asty

res = pd.DataFrame(results)
base_fp = res.loc[0, "false_positives"]
res["false_positive_reduction_vs_rules"] = 1 - res.false_positives / base_fp
res
```

Out[6]:

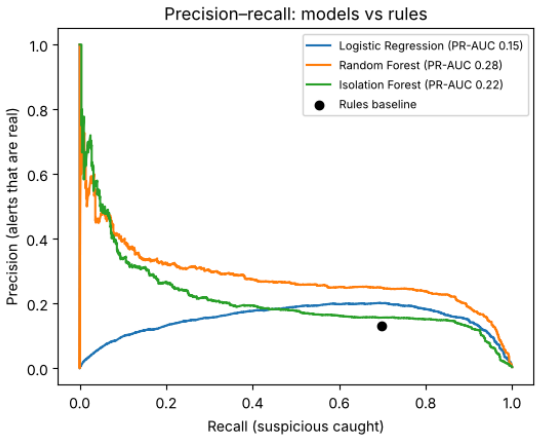
	model	precision	recall	alerts	false_positives	PR_AUC	ROC_AUC	false_positive_re
0	Rules baseline	0.130	0.697	3867	3365	NaN	NaN	
1	Logistic Regression (same recall as rules)	0.202	0.697	2489	1987	0.152	0.986	
2	Random Forest (same recall as rules)	0.249	0.697	2013	1511	0.282	0.990	
3	Isolation Forest (same recall as rules)	0.157	0.697	3206	2704	0.216	0.981	

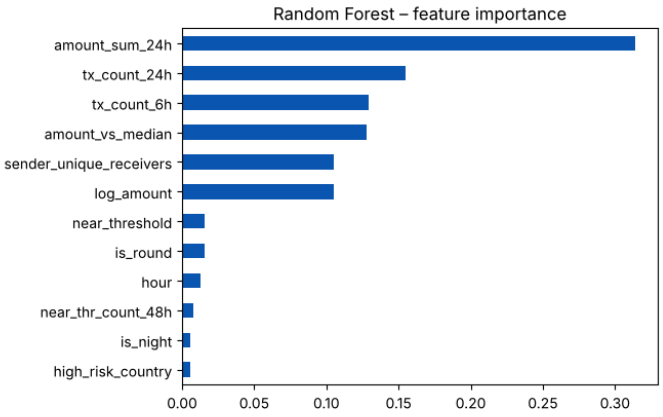
In [7]:
```
best = res.iloc[1:].sort_values("false_positives").iloc[0]
print(f"Rules baseline: recall {res.loc[0,'recall']:.1%}, {base_fp:,} false positiv
print(f"Best model: {best.model} – same recall, {best.false_positives:,} false posi
      f"({best.false_positive_reduction_vs_rules:.1%} fewer), precision {best.preci
```

```
Rules baseline: recall 69.7%, 3,365 false positives
Best model: Random Forest (same recall as rules) – same recall, 1,511 false positive
s (55.1% fewer), precision 24.9%, PR-AUC 0.282
```

In [8]:
```
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
for name, s in [("Logistic Regression", lr_score), ("Random Forest", rf_score), ("I
    p, r, _ = precision_recall_curve(y_test, s)
    ax[0].plot(r, p, label=f"{name} (PR-AUC {average_precision_score(y_test, s):.2f
ax[0].scatter([rules_recall], [precision_score(y_test, r_test)], color="black", zor
ax[0].set_xlabel("Recall (suspicious caught)"); ax[0].set_ylabel("Precision (alerts
ax[0].set_title("Precision–recall: models vs rules"); ax[0].legend(fontsize=8)

imp = pd.Series(rf.feature_importances_, index=FEATURES).sort_values()
imp.plot.barh(ax=ax[1], color="#0E57B2"); ax[1].set_title("Random Forest – feature
plt.tight_layout(); plt.savefig("aml_results.png", dpi=130); plt.show()
```





6. Conclusions

- Behavioural features (velocity in 24h/6h windows, repeated near-threshold amounts, deviation from the account's own normal amount, counterparty breadth) carry most of the signal – the same logic AML analysts use manually.
- At the **same detection rate as the rules baseline**, the supervised models raise **far fewer false alerts**, which directly reduces investigator workload.
- Isolation Forest finds anomalies **without labels**, useful where confirmed SAR outcomes are scarce, but it is noisier than supervised models.
- Next steps for production: time-based train/test split, threshold tuning per risk segment, graph features on the transaction network, model monitoring and validation documentation.